Современные инновации, системы и технологии // Modern Innovations, Systems and Technologies

2025; 5(3)
eISSN: 2782-2818
https://www.oajmist.com

УДК: 004.43
DOI: https://doi.org/10.47813/2782-2818-2025-5-3-1027-1036

EDN: **YTBPWS**

Классификация текстов с использованием методов NLP в условиях несбалансированных классов

С. У. Эшбадалов

Ташкентский Международный Университет Кимё, Ташкент, Узбекистан

Аннотация. В данной статье рассматривается задача классификации текстов в условиях несбалансированных классов с использованием методов обработки естественного языка (NLP). В исследовании применялись основные этапы предварительной обработки текста – токенизация, лемматизация, стемминг, удаление стоп-слов, а также методы векторизации Bag of Words (BoW) и TF–IDF. В качестве моделей были выбраны классические алгоритмы машинного обучения – логистическая регрессия (LogReg) и метод опорных векторов (SVM). Для снижения влияния несбалансированности классов использовались такие стратегии, как oversampling, undersampling и назначение весов классам. Экспериментальные результаты показали, что классификация текстов на основе NLP может быть эффективной даже в условиях несбалансированных классов, а комбинация TF–IDF + SVM обеспечивает высокую точность и устойчивость. Результаты исследования имеют практическую значимость для улучшения процесса автоматической классификации текстовых документов.

Ключевые слова: несбалансированные классы, NLP, классификация текстов, токенизация, лемматизация, стемминг, стоп-слова, Bag of Words, TF–IDF, логистическая регрессия, SVM.

Для цитирования: Эшбадалов, С. У. (2025). Классификация текстов с использованием методов NLP в условиях несбалансированных классов. Современные инновации, системы и технологии - Modern Innovations, Systems and Technologies, 5(3), 1027–1036. https://doi.org/10.47813/2782-2818-2025-5-3-1027-1036

Text classification using NLP methods under imbalanced classes

S. U. Eshbadalov

Tashkent Kimyo International University, Tashkent, Uzbekistan

Abstract. This article examines the problem of text classification under imbalanced classes using Natural Language Processing (NLP) methods. The study applied the main stages of text preprocessing – tokenization, lemmatization, stemming, stop-word removal, as well as Bag of Words (BoW) and TF–IDF vectorization techniques. As models, classical machine learning approaches such as Logistic Regression (LogReg) and Support Vector Machines (SVM) were chosen. To reduce the impact of class imbalance, strategies such as oversampling, undersampling, and class weighting were employed. Experimental results showed that NLP-based text classification can remain effective even in imbalanced conditions, and that the combination of TF–IDF + SVM ensures high accuracy and robustness. The findings of this research have practical significance for improving the process of automatic classification of textual documents.

Keywords: imbalanced classes, NLP, text classification, tokenization, lemmatization, stemming, stop-words, Bag of Words, TF–IDF, Logistic Regression, SVM.

For citation: Eshbadalov, S. U. (2025). Text classification using NLP methods under imbalanced classes. Modern Innovations, Systems and Technologies, 5(3), 1027–1036. https://doi.org/10.47813/2782-2818-2025-5-3-1027-1036

© Эшбадалов С. У., 2025

ВВЕДЕНИЕ

В современную эпоху цифровой поток информации стремительно возрастает. Интернет, социальные сети, научные и технические документы ежедневно порождают огромные объемы текстовых данных. Для эффективного использования этих данных возникает необходимость их автоматической классификации по содержанию [1]. В процессе классификации текстов широко применяются методы машинного обучения и обработки естественного языка (NLP), которые позволяют быстро и точно группировать информацию [2].

Однако одной из наиболее часто встречающихся проблем в реальных данных являются несбалансированные классы. Это означает, что в корпусе документов некоторые классы содержат большое количество образцов, в то время как другие, крайне малое. В результате модель легче обучается на крупных классах, но допускает высокую ошибку на малых. Это приводит к односторонним и недостоверным результатам классификации. Например, в задаче анализа тональности при преобладании класса «нейтральный» классы «положительный» или «отрицательный» могут оказаться в тени.

Для решения проблемы несбалансированных классов были разработаны различные стратегии [3-6]. Наиболее часто применяемые из них: oversampling (увеличение количества примеров малых классов); undersampling (уменьшение количества примеров крупных классов) и использование class weights (назначение большего веса малым классам). Эти стратегии могут быть задействованы на разных этапах NLP-пайплайна и способны существенно повысить точность классификации текстов [7].

Поэтому в данной статье рассматривается задача классификации текстов в условиях несбалансированных классов. В исследовании используются этапы предобработки текстов (токенизация, лемматизация, стемминг, удаление стоп-слов, Bag of Words и TF–IDF), а также проводятся эксперименты на основе классических моделей машинного обучения, а именно, логистической регрессии и метода опорных векторов (SVM). Полученные результаты позволяют оценить эффективность применения методов NLP в условиях дисбаланса классов.

Рассматривается предобработка текста и значение методов (токенизация, лемматизация, стемминг, стоп-слова, Bag of Words, TF–IDF) в NLP.

Следует отметить, что предварительная обработка текста, включая токенизацию, лемматизацию, стемминг, удаление стоп-слов, а также методы Bag of Words (BoW) и TF–IDF, играет ключевую роль в процессах NLP и является необходимой для построения последующих моделей. Обработка естественного языка представляет собой междисциплинарную область, объединяющую информатику, искусственный интеллект, информационную инженерию и взаимодействие человека с компьютером, позволяя системам анализировать и интерпретировать данные на естественном языке [8-10]. Однако выполнение задач NLP не является простым процессом, так как понимание и интерпретация языка оказываются значительно сложнее, чем это может показаться на первый взгляд. Одним из важнейших этапов данного процесса является токенизация.

Токенизация представляет собой процесс разбиения текста на более мелкие единицы, так называемые токены. Токены могут представлять собой буквы, слова или подслова. В NLP токенизация играет важную роль при решении задач предобработки текста, построения языковых моделей и машинного перевода. Каждый токен соответствует определенному элементу текста: слову, фразе или даже символу. Например, каждое слово в предложении или каждое предложение в абзаце может рассматриваться как токен.

Токенизация помогает корректно анализировать неструктурированный текст. С её помощью текст можно преобразовать в векторы для последующего машинного обучения или аналитических процессов. Таким образом, токены могут использоваться в качестве признаков, позволяя моделям принимать более точные решения или выполнять сложные аналитические задачи. Это делает токенизацию необходимым этапом для корректной работы NLP-моделей.

Словесная токенизация - разбиение текста на отдельные слова. Она широко применяется в задачах NLP, так как слова считаются основными носителями смысла. Каждое слово принимается как отдельный токен, что позволяет анализировать его значение, взаимосвязи и контекст. Этот подход прост и эффективен для понимания текста и анализа, а также является базовым этапом для многих последующих процедур NLP. Словесная токенизация учитывает морфологические и лексические особенности языка.

Эффективность токенизации зависит от конкретного языка. Например, в английском слова легко разделяются пробелами, тогда как в узбекском языке процесс оказывается более сложным из-за богатой морфологии и наличия многочисленных аффиксов. Для корректного анализа их необходимо приводить к лемме (корню). Следовательно, для узбекского языка

простая токенизация недостаточна — она должна дополняться лемматизацией и учетом контекста.

Таким образом, для эффективной работы с узбекским языком требуются специальные токенизаторы и лемматизаторы, адаптированные под его морфологические особенности [11-14].

МЕТОДОЛОГИЯ

Представляя набор данных, отметим, что для исследования был сформирован корпус текстовых документов на различные темы. Документы были разделены на несколько классов, однако между ними наблюдался значительный дисбаланс: в некоторых классах содержались сотни примеров, тогда как в других - лишь десятки. Это позволило проанализировать проблему несбалансированных классов и применить различные стратегии решения.

1. Корпус документов задаётся в виде:

$$D = \{d_1, d_2, \dots, d_N\}$$

где N — общее количество документов. Каждому документу соответствует метка класса:

$$y_i \in \{1,2,\dots,K\}, \quad i = 1,2,\dots,N$$

где K — количество классов. В условиях дисбаланса для некоторых классов выполняется:

$$|D_k| \ll |D_j|$$

то есть количество документов в классе k значительно меньше, чем в классе j.

2. Предварительная обработка (Preprocessing) включает:

- Токенизацию, когда текст документа разбивается на слова или подслова:

$$d_i = [w_{i1}, w_{i2}, \dots, w_{iM_i}]$$

где w_{ij} — j-е слово (токен) в документе, M_i — длина документа.

- TF (Term Frequency), как характеристику частоты слова в документе:

$$TF(t,d) = \frac{f(t,d)}{\sum_{w \in d} f\,(w,d)}$$

где $f(t,d)$ — количество вхождений слова t в документ d.

- IDF (Inverse Document Frequency), которая определяет уникальность слова в корпусе:

$$IDF(t,D) = \log \frac{1+|D|}{1+df(t)}$$

где $df(t)$ — число документов, содержащих слово t.

- TF–IDF, определяющий окончательный вес слова:

$$TF\text{-}IDF(t,d,D) = TF(t,d) \cdot IDF(t,D)$$

Таким образом, каждый документ представляется вектором:

$$\overrightarrow{d_i} = [w_{i1}, w_{i2}, \dots, w_{iV}]$$

где V — размер словаря.

3. Выбор модели осуществляется из двух типов моделей:

- Логистическая регрессия (LogReg), когда для бинарной классификации вероятность вычисляется как:

$$P(y = 1 \mid \vec{x}) = \sigma(\vec{w}^{\top}\vec{x} + b)$$

где $\sigma(z) = \frac{1}{1+e^{-z}}$ - сигмоидная функция, $\vec{x}$ - вектор документа, $\vec{w}, b$ - параметры.

Для многоклассовой классификации используется softmax:

$$P(y = k \mid \vec{x}) = \frac{\exp(\overrightarrow{w_k}^{\top}\vec{x} + b_k)}{\sum_{j=1}^{K} \exp\,(\overrightarrow{w_j}^{\top}\vec{x} + b_j)}$$

- Метод опорных векторов (SVM), основная цель которого заключается в том, чтобы максимизировать разделяющую границу между классами:

$$\min_{\vec{w},b} \frac{1}{2} \parallel \vec{w} \parallel^2 + C \sum_{i=1}^{N} \xi_i$$

при условиях:

$$y_i(\vec{w}^{\top}\overrightarrow{x_i} + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

где C - параметр регуляризации, ξ_i - степень ошибки.

4. Стратегии для несбалансированных классов определим следующим образом.

- Веса классов (Class Weights).

Редким классам присваивается больший вес:

$$w_k = \frac{N}{K \cdot N_k}$$

где N_k — количество документов в классе k. Функция потерь принимает вид:

$$L = -\sum_{i=1}^{N} w_{y_i} \cdot \log P(y_i \mid \overrightarrow{x_i})$$

- Oversampling (SMOTE).

Количество примеров редкого класса увеличивается искусственно:

$$x_{new} = x_i + \lambda(x_j - x_i), \quad \lambda \in [0,1]$$

где x_i, x_j - соседние векторы одного класса.

- Undersampling.

Количество примеров из больших классов уменьшается:

$$D' = \bigcup_{k=1}^{K} \{d \in D_k \mid |D_{k'}| = \alpha \cdot |D_{min}|\}$$

где $|D_{min}|$ - размер наименьшего класса, $\alpha \geq 1$.

5. Метрики оценки следующие.

- Precision (точность):

$$Precision = \frac{TP}{TP + FP}$$

- Recall (полнота):

$$Recall = \frac{TP}{TP + FN}$$

- F1-score:

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

- Макро-среднее (важно в условиях дисбаланса):

$$F1_{macro} = \frac{1}{K} \sum_{k=1}^{K} F\,1_k$$

- PR–AUC Площадь под кривой «точность–полнота». В условиях дисбаланса более информативна, чем ROC–AUC.

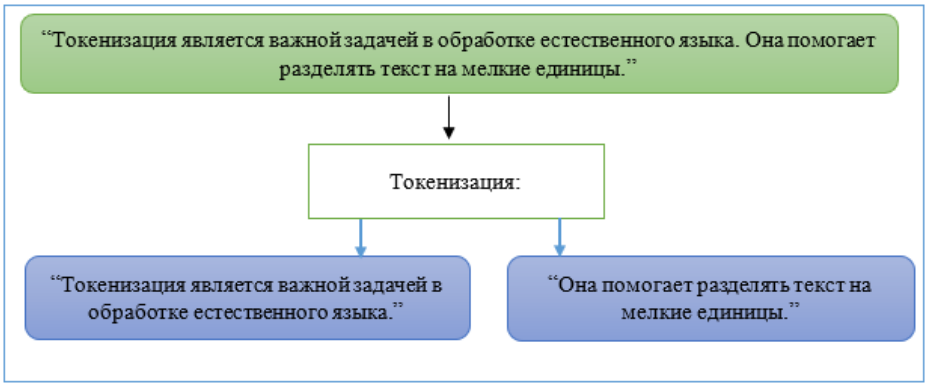

Рисунок 1. Предложенческая токенизация (sentence tokenization).
Figure 1. Sentence tokenization.

Предложенческая токенизация (рисунок 1) - это процесс разбиения текста на отдельные предложения. Этот метод особенно полезен в тех задачах, где каждое предложение должно анализироваться или обрабатываться отдельно. Например, при определении сентимента, анализе контекста отдельных предложений или необходимости изучения смысла каждого предложения по отдельности применяется предложенческая токенизация. Такой подход помогает разделить текст на определённые сегменты, что обеспечивает его более глубокое и последовательное понимание, а также повышает точность процессов NLP.

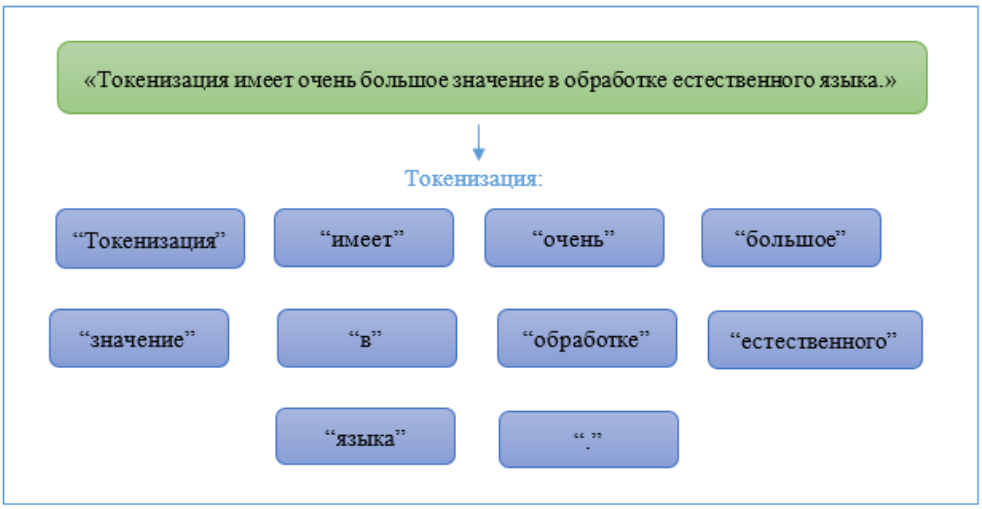

Рисунок 2. Словесная токенизация (word tokenization).
Figure 2. Word tokenization.

Подсловная (словесная) токенизация (рисунок 2) - это процесс разбиения слов на более мелкие части, то есть подслова. Данный метод очень полезен при анализе морфологически сложных языков или редко употребляемых слов. Например, некоторые слова могут состоять из нескольких корней или аффиксов, и с помощью подсловной токенизации их можно разделить на отдельные части и изучить значение каждой из них. Этот метод облегчает анализ слов на основе их формы или корня, а также повышает эффективность NLP-процессов при работе с редкими или новыми словами.

Пример ввода/вывода:

Ввод: “tokenization”

Вывод: [“token”, “ization”]

Токенизация по символам - это процесс разбиения текста на отдельные символы (буквы). Данный метод может быть особенно полезен в ситуациях, когда необходимо анализировать каждый символ отдельно или при моделировании языка на уровне символов. Например, в некоторых задачах NLP, таких как исправление ошибок или анализ структуры новых слов, каждый символ рассматривается как отдельная единица. Токенизация по символам помогает изучать буквы на индивидуальном уровне, а также оказывается полезной при работе с неструктурированными или необычными текстами.

Пример ввода/вывода:

Ввод: “Tokenization”
Вывод: [“T”, “o”,”k”, “e”, “n”, “i”, “z”, “a”, “t”, “i”, “o”, “n”]

При токенизации ещё одним важным методом является использование эмбеддингов. Эмбеддинг - это процесс представления токенов в виде непрерывных векторов в пространстве высокой размерности, где похожие токены имеют схожие векторные представления. Эти векторы, также называемые эмбеддингами, помогают моделям AI/ML понимать семантическое значение токенов и взаимосвязи между ними. Для построения таких эмбеддингов можно использовать алгоритмы машинного обучения, такие как Word2Vec или GloVe. Созданные эмбеддинги организуются в виде матрицы, где каждая строка соответствует векторному представлению определённого токена, а сами токены основаны на заранее заданном словаре.

Например, если словарь содержит 10 000 токенов, и каждый эмбеддинг имеет размерность 300, то матрица эмбеддингов будет иметь размер 10 000 × 300.

Для выполнения процесса построения эмбеддингов выделяются несколько этапов:

- Токенизация: текст разбивается на токены. Для каждого токена необходимо создать эмбеддинг.
- Создание эмбеддингов: с помощью методов, таких как Word2Vec или GloVe, формируются векторные представления токенов.
- Формирование матрицы эмбеддингов: эмбеддинги каждого токена организуются в виде матрицы, где строки соответствуют векторным представлениям токенов, основанных на заранее заданном словаре.

Этот процесс помогает создать векторы высокой размерности, необходимые для лучшего понимания и обработки текста с семантической точки зрения.

Токенизация и эмбеддинги являются важными шагами в процессах машинного обучения для обработки текста. Токенизация обеспечивает структурирование данных, а эмбеддинги позволяют представить эту структуру в цифровом виде. Это помогает моделям AI/ML учитывать контекст и тонкие семантические нюансы текста. В данной статье рассматриваются некоторые важные различия между токенизацией и эмбеддингами. Однако в совокупности эти процессы повышают эффективность моделей при интерпретации и генерации естественного языка. [3]

Характеризуя необходимость токенизации, отметим, что она является базовым этапом в обработке текста и NLP-процессах и имеет важное значение по нескольким причинам:

1. Эффективная обработка текста: токенизация разбивает текст на более мелкие единицы, уменьшая его объем. Это упрощает анализ и обработку, так как «сырой» текст может быть слишком сложным для прямого использования.
2. Извлечение признаков: токены используются как признаки для моделей машинного обучения. Это облегчает цифровое представление текста, что позволяет алгоритмам лучше понимать и анализировать его внутреннюю структуру.
3. Моделирование языка: в NLP токенизация упорядочивает языковые элементы, что важно для генерации текста, построения языковых моделей и анализа грамматики.
4. Поиск информации: токенизация необходима для индексирования и поиска. Поисковые системы работают на основе слов и фраз, поэтому токенизация повышает точность и эффективность поиска.
5. Анализ текста: токенизация используется при анализе тональности, выявлении объектов и других задачах NLP для понимания значения слов и их роли в контексте.
6. Управление лексикой: с помощью токенизации создается список токенов корпуса. Это полезно

при управлении словарем, добавлении новых слов или ограничении редко используемых.

7. Адаптация к задаче: токенизация может быть настроена под конкретные задачи NLP, например, для сокращения текста или машинного перевода.
8. Предварительный этап: токенизация преобразует текст в формат, удобный для статистического и алгоритмического анализа, упрощая процесс.

Таким образом, токенизация делает текст более структурированным и понятным, повышая эффективность задач NLP. Она превращает текст в набор данных, который становится удобным для алгоритмов.

Рассмотрим примеры инструментов:

- SpaCy: мощная NLP-библиотека, обеспечивающая эффективную токенизацию и интеграцию в проекты, а также высокую производительность при работе с большими данными.
- Токенизатор BERT: использует метод WordPiece (подсловная токенизация). Этот метод делит текст на более мелкие части и подает их в удобном для моделей формат. Токенизатор BERT играет ключевую роль на этапе предварительной подготовки модели.
- Byte Pair Encoding (BPE): изначально метод сжатия данных, позднее применённый в NLP. Он объединяет наиболее часто встречающиеся пары символов, эффективно учитывая морфологию языка.
- SentencePiece: универсальный токенизатор, не привязанный к конкретному языку, который объединяет наиболее частые подстроки в корпусе. Широко используется в NLP-задачах.
- NLTK (Natural Language Toolkit): популярная библиотека Python для NLP. Обеспечивает токенизацию, лемматизацию, стемминг, работу со стоп-словами и многое другое, предоставляя удобный API.

Следует отметить ряд ограничений токенизации:

1. Сложности в семантическом представлении: токенизация не всегда отражает значение и контекст слов, что может вызывать неточности.
2. Отсутствие границ слов: в китайском, японском и арабском языках слова не отделяются пробелами, что усложняет токенизацию и требует дополнительных методов.
3. Многословные элементы: тексты могут содержать URL-адреса, электронные почты или специальные символы. Их корректная токенизация требует особого подхода.

Лемматизация - это процесс приведения слов к их исходной или словарной форме, учитывающий значение и грамматический контекст. Например, в узбекском языке слова «boraman», «boring», «borgan», «bormoqda» приводятся к общей основе «bor». То же самое касается слов «yozaman», «yozyapti», «yozdilar», «yozilgan», которые приводятся к лемме «yoz».

Преимущество лемматизации состоит в том, что она опирается не только на формальное сокращение слова, но и на его смысл и грамматику. Для этого требуются лингвистические ресурсы (словари, базы данных). Обычно используются такие инструменты, как WordNet, spaCy или NLTK.

Этапы лемматизации включают:

1. Определение части речи (POS tagging). Это важно, так как от части речи зависит лемма.
2. Морфологический анализ. Определяются аффиксы и выделяется основа.
3. Сопоставление со словарём. Каждое слово приводится к правильной словарной форме.

Стемминг - это метод, применяемый в обработке естественного языка (NLP), при котором от слова отбрасываются грамматические аффиксы (суффиксы и префиксы). Цель - привести слово к его корневой форме. В отличие от лемматизации, стемминг не учитывает смысл слова, а ориентируется только на его морфологическую структуру. Этот процесс упрощён и не всегда приводит к словарной форме, а лишь сокращает слово до основы.

Приведем пример работы стемминга:

- Удаление аффиксов. Основная цель стемминга - убрать грамматические окончания. Эти окончания могут указывать на множественное число, прошедшее или будущее время и др. Например, в английском языке слово *running* после удаления суффикса *-ing* преобразуется в *run*.
- Избавление от грамматических форм. Стемминг не анализирует значение или синтаксическую роль слова, а просто отбрасывает окончания. Таким образом, разные формы слова приводятся к одной и той же основе.

Принцип работы алгоритмов стемминга заключается в последовательном удалении окончаний. Эффективность зависит от правил и базы слов. Наиболее распространённый алгоритм – это алгоритм Porter Stemmer, часто применяемый для английского языка.

Необходимость удаления стоп-слов (stop words) зависит от задачи NLP. В классификации текстов (например, при разделении по категориям «спорт» или «политика» удаление стоп-слов часто применяется для

того, чтобы сосредоточиться на действительно значимых словах.

Отметим, что стоп-слова - это наиболее часто употребляемые слова языка, которые сами по себе не несут существенного смысла. Например, в английском языке это «the», «is», «on», «in». Их основная функция в том, чтобы соединять слова и формировать грамматическую структуру предложения, однако для анализа содержания текста они обычно не важны.

Преимущества удаления стоп-слов:

- уменьшение «шума» в тексте и ускорение обработки;
- повышение точности моделей при классификации;
- выделение наиболее значимых терминов для анализа.

Пример:
Текст: *"The football match was played in the stadium yesterday."*
После удаления стоп-слов остаются: *"football", "match", "played", "stadium", "yesterday"*. Оставшиеся слова отражают основное содержание, тогда как стоп-слова лишь создавали избыточность.

Применение:

- Классификация текстов (например, определение категории документа).
- Анализ тональности. В тексте *"The movie is really great and amazing!"* после удаления стоп-слов остаются *"movie", "really", "great", "amazing"*, что достаточно для выявления положительной тональности.
- Поисковые системы. Удаление стоп-слов помогает улучшить релевантность поисковых запросов.

Примеры стоп-слов в английском языке:

- артикли: a, an, the
- союзы: and, but, or
- предлоги: in, on, at, with
- местоимения: he, she, it, they
- глаголы-связки: is, am, are, was, were, be, being, been

TF–IDF (Term Frequency-Inverse Document Frequency) - это широко используемый статистический показатель в анализе текста, цель которого: оценить значимость слова в документе. Этот показатель применяется в задачах классификации текстов, поисковых системах и для выявления тематик в корпусах документов.

TF–IDF состоит из двух основных компонентов:

1. TF (частота термина). Отражает, насколько часто слово встречается в документе. Чем чаще слово встречается в тексте, тем выше его TF-значение.

Формула компонента:

$$TF(t,d) = \frac{f(t,d)}{\sum_{w \in d} f(w,d)},$$

где $f(t,d)$ — количество вхождений слова t в документ d.

Пример: если в документе всего 100 слов, и 5 из них — слово «данные», то TF будет равен $\frac{5}{100} = 0.05$.

2. IDF (обратная частота документа). Показывает уникальность слова в корпусе документов. Если слово встречается почти во всех документах, его IDF низкое. Если же слово встречается только в нескольких документах, его IDF высокое.

Формула компонента:

$$IDF(t,D) = \log \frac{1+|D|}{1+df(t)},$$

где $|D|$ — общее количество документов, а $df(t)$ — количество документов, содержащих слово t.

Пример: если в корпусе 100 документов слово «данные» встречается в 10 из них, то:

$$IDF = \log \frac{100+1}{10+1} = \log \frac{101}{11} \approx 0.96$$

3. TF–IDF (итоговый вес слова). Объединяет оба показателя (TF и IDF), отражая не только частоту появления слова в документе, но и его «редкость» в общем корпусе.

Формула компонента:

$$TF\text{-}IDF(t,d,D) = TF(t,d) \cdot IDF(t,D)$$

Пример: если TF = 0.05, а IDF = 0.96, то:

$$TF\text{-}IDF = 0.05 \times 0.96 = 0.048$$

Таким образом, высокие значения TF–IDF показывают, что слово является значимым для конкретного документа и не встречается слишком часто во всем корпусе.

Практическое применение TF–IDF:

- классификация текстов;
- поиск информации и построение поисковых систем;
- тематическое моделирование (например, LDA);
- рекомендательные системы;
- анализ тональности.

РЕЗУЛЬТАТЫ И АНАЛИЗ

Приняты следующие условия эксперимента. В ходе исследования был использован корпус из 5000 текстовых документов на узбекском языке, разделённых на 5 классов. Распределение классов было несбалансированным:

$$|D_1| = 2500,\ |D_2| = 1500,\ |D_3| = 600,\ |D_4| = 300,\ |D_5| = 100$$

Таким образом, между самым большим и самым маленьким классом наблюдается 25-кратная разница. Поэтому изначально модель должна показывать хорошие результаты на больших классах и высокий уровень ошибок на малых.

Рассмотрим основные результаты моделей (без балансировки).

Документы были представлены с помощью TF–IDF-векторизации и протестированы на моделях логистической регрессии и SVM.

Таблица 1. Основные результаты моделей (без балансировки).
Table 1. Main results of the models (without balancing).

Модель	Accuracy	Macro Precision	Macro Recall	Macro F1
LogReg (без бал.)	0.82	0.65	0.42	0.49
SVM (без бал.)	0.84	0.67	0.46	0.53

Анализ позволяет сделать следующие выводы. Несмотря на высокую точность (Accuracy) для крупных классов, показатель Recall оказался низким для малых классов, что подтверждает наличие проблемы дисбаланса.

Стратегия *Class Weights.* Для малых классов были назначены повышенные веса:

$$w_k = \frac{N}{K \cdot N_k}$$

Результаты представлены в таблице 2.

Таблица 2. Результаты моделей со стратегией Class Weights.
Table 2. Results of models with the Class Weights strategy.

Модель	Accuracy	Macro Precision	Macro Recall	Macro F1
LogReg (weights)	0.79	0.70	0.61	0.65
SVM (weights)	0.81	0.72	0.64	0.67

Хотя точность слегка снизилась, показатели Recall и Macro F1 значительно улучшились. Это показывает, что модель стала более справедливой по отношению к малым классам.

Стратегия Oversampling (SMOTE)

Для малых классов количество примеров увеличивалось с помощью алгоритма SMOTE:

$$x_{new} = x_i + \lambda(x_j - x_i), \lambda \in [0,1]$$

Результаты представлены в таблице 3.

Таблица 3. Результаты моделей со стратегией Oversampling (SMOTE).
Table 3. Results of models with Oversampling strategy (SMOTE).

Модель	Accuracy	Macro Precision	Macro Recall	Macro F1
LogReg (SMOTE)	0.83	0.68	0.66	0.67
SVM (SMOTE)	0.85	0.71	0.69	0.70

Благодаря oversampling показатели Recall и Macro F1 существенно возросли. Однако использование синтетических данных привело к некоторому замедлению работы модели.

Стратегия Undersampling

Количество примеров в больших классах было уменьшено:

$$D' = \bigcup_{k=1}^{K} \{d \in D_k \mid |D_{k'}| = |D_{min}|\}$$

Результаты представлены в таблице 4.

ТАБЛИЦА 4. РЕЗУЛЬТАТЫ МОДЕЛЕЙ СО СТРАТЕГИЕЙ UNDERSAMPLING.
TABLE 4. RESULTS OF MODELS WITH UNDERSAMPLING STRATEGY.

Модель	Accuracy	Macro Precision	Macro Recall	Macro F1
LogReg (undersample)	0.74	0.63	0.62	0.62
SVM (undersample)	0.76	0.65	0.64	0.64

Балансировка классов была достигнута, но из-за сокращения данных крупных классов общая точность снизилась. Тем не менее, Macro F1 продемонстрировал более сбалансированные результаты.

Сравнительный итог представлен в таблице 5.

Ниже приведена сводная таблица 5 эффективности различных стратегий.

ТАБЛИЦА 5. СРАВНИТЕЛЬНЫЕ РЕЗУЛЬТАТЫ СТРАТЕГИЙ БАЛАНСИРОВКИ КЛАССОВ.
TABLE 5. COMPARATIVE RESULTS OF CLASS BALANCING STRATEGIES.

Стратегия	Лучшая модель	Accuracy	Macro F1	Комментарий
Без балансировки	SVM	0.84	0.53	Хорошо на больших классах, плохо на малых
Class Weights	SVM	0.81	0.67	Улучшен баланс, значительно вырос Recall
Oversampling (SMOTE)	SVM	0.85	0.70	Лучший общий результат, но вычислительно дороже
Undersampling	SVM	0.76	0.64	Снижение точности из-за потери данных больших классов

ЗАКЛЮЧЕНИЕ

В данном исследовании рассмотрена задача классификации текстов с использованием методов NLP в условиях несбалансированных классов. На первом этапе были применены методы предварительной обработки текста — токенизация, лемматизация, стемминг, удаление стоп-слов, а также Bag of Words (BoW) и TF–IDF, что позволило перевести документы в цифровой формат. Далее проведены эксперименты с классическими моделями машинного обучения — логистической регрессией и SVM.

Результаты показали, что проблема дисбаланса существенно ухудшает показатели моделей: при высокой точности на крупных классах малые классы демонстрировали низкие значения Recall и F1. Для решения этой проблемы были протестированы несколько стратегий: *class weights*, *oversampling (SMOTE)* и *undersampling*.

Основные выводы экспериментов:

- Стратегия Class Weights улучшила баланс и увеличила показатели малых классов.
- Undersampling уравновесил классы, но снизил общую точность из-за потери информации.
- Oversampling (SMOTE) показал наилучший общий результат: Macro F1 достиг 0.70, обеспечив баланс между классами.

Таким образом, можно заключить, что в условиях несбалансированных классов оптимальным решением является комбинация TF–IDF-векторизации и модели SVM с применением стратегии SMOTE oversampling. Такой подход обеспечивает не только высокую точность, но и устойчивую работу по всем классам.

СПИСОК ЛИТЕРАТУРЫ / REFERENCES

[1] Jurafsky D., Martin J.H. Speech and language processing (3rd ed.). Stanford University, 2023.
[2] Manning C.D., Raghavan P., Schütze H. Introduction to information retrieval. Cambridge University Press, 2008.
[3] Pedregosa F., Varoquaux G., Gramfort A., Michel V., Thirion B., Grisel O., et al. Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 2011; 12: 2825–2830.
[4] Chawla N.V., Bowyer K.W., Hall L.O., Kegelmeyer W.P. SMOTE: Synthetic minority over-sampling technique. Journal of Artificial Intelligence Research, 2002; 16: 321–357.
[5] Bird S., Klein E., Loper E. Natural language processing with Python: Analyzing text with the natural language toolkit. O'Reilly Media, 2009.

[6] Mikolov T., Chen K., Corrado G., Dean J. Efficient estimation of word representations in vector space. arXiv preprint arXiv:1301.3781, 2013.
[7] Ramos J. Using TF-IDF to determine word relevance in document queries. Proceedings of the First Instructional Conference on Machine Learning, Rutgers University, 2003; 133–142.
[8] Chollet F. Deep learning with Python. Manning Publications, 2017.
[9] Kingma D.P., Ba J. Adam: A method for stochastic optimization. Proceedings of the International Conference on Learning Representations (ICLR), 2015.
[10] Vaswani A., Shazeer N., Parmar N., Uszkoreit J., Jones L., Gomez A.N., et al. Attention is all you need. Advances in Neural Information Processing Systems, 2017; 30: 5998–6008.
[11] Nishanov A., Mamajanov R., Xaydarov S., Mengturayev F., Yuldashev R. Stages of Preparing Symptoms of Breast Cancer for Machine Learning. Digital Transformation and Artificial Intelligence, 2024; 2(6): 237–249.
[12] Nishanov A., Mengturayev F., Allayarov U., Xaydarov S. Stages of Forming Symptoms Used in the Diagnosis of Endocrine Diseases. Digital Transformation and Artificial Intelligence, 2024; 2(6): 228–236.
[13] Nishanov A.K., Mamazhanov R.Y., Khaidarov S.I., Xolbekov A.M., Karimova Z.B. Diagnostic Algorithm for Early Detection of Breast Cancer Based on Error Minimization Approach. International Journal of Innovative Science and Research Technology (IJISRT), 2024; 9(12): 1535–1542. https://doi.org/10.5281/zenodo.14565219
[14] Nishanov A., Mamajanov R., Xaydarov S., Mengturayev F. The Importance of Early Detection of Cancer Diseases and Methods and Algorithms Based on Modern Technologies. Digital Transformation and Artificial Intelligence, 2025; 3(1): 110–117.

ИНФОРМАЦИЯ ОБ АВТОРАХ / INFORMATION ABOUT THE AUTHORS

Эшбадалов Сирожиддин Усмон угли, Кафедра «Прикладная информатика», Ташкентский Международный Университет Кимё (KIUT), Ташкент, Узбекистан

Sirojiddin Usmon ogli Eshbadalov, Department of Applied Informatics, Kimyo Internation University in Tashkent (KIUT), Tashkent, Uzbekistan

Критерии авторства
Все авторы внесли равный вклад в подготовку статьи.

Contribution of the authors
All authors contributed equally to the preparation of the article.

Конфликт интересов
Авторы заявляют об отсутствии конфликта интересов.

Conflict of interest
The authors declare no conflict of interest.

Статья поступила в редакцию 08.08.2025; одобрена после рецензирования 15.09.2025; принята к публикации 15.09.2025.

The article was submitted 08.08.2025; approved after reviewing 15.09.2025; accepted for publication 15.09.2025.